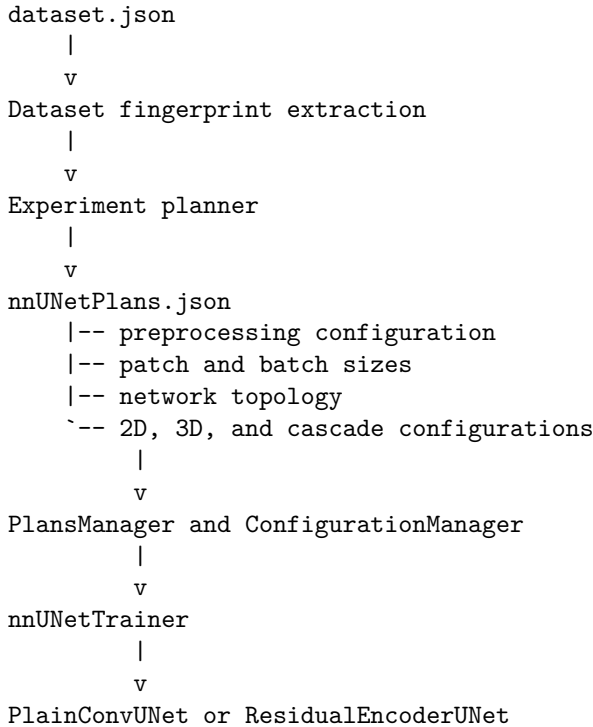


nnU-Net Architecture Notes

Configuration, Network Construction, and Neural Network Layers

nnU-Net is best understood as a **configuration-generating system around a U-Net**, rather than as one fixed neural network. It examines the dataset, creates a dataset-specific plan, and constructs the preprocessing and network topology described by that plan.

1. End-to-End Architecture



The central idea is:

The fingerprint describes the dataset, the planner converts that description into a plans file, and the trainer constructs the exact U-Net encoded in that plans file.

2. The Configuration System

Several different files and objects can be mistaken for "the configuration." They have distinct responsibilities.

2.1 Global Runtime Configuration

`nnunetv2/configuration.py` contains only a few global runtime defaults:

- Number of preprocessing processes
- Number of data-augmentation processes
- Anisotropy threshold

It does not define the neural network.

2.2 Dataset Description: `dataset.json`

The dataset description includes:

- Image channels or modalities
- Labels or regions
- File extension
- Number of training cases

The planner and label manager use it to determine the network's input and output channels.

2.3 Dataset Fingerprint: `dataset_fingerprint.json`

The fingerprint is generated by:

`nnunetv2/experiment_planning/dataset_fingerprint/fingerprint_extractor.py`

It summarizes properties such as:

- Image spacing
- Image shape after cropping
- Foreground intensity statistics
- Relative size retained after cropping
- Approximate dataset size

The fingerprint is the empirical description from which nnU-Net derives its plans.

2.4 Plans File: `nnUnetPlans.json`

The plans file is the main persistent configuration. It commonly contains named configurations such as:

- 2d
- 3d_fullres
- 3d_lowres
- 3d_cascade_fullres

Each configuration controls both preprocessing and training. A simplified example is:

```
{
  "spacing": [2.5, 0.8, 0.8],
  "patch_size": [64, 192, 192],
  "batch_size": 2,
  "normalization_schemes": ["CTNormalization"],
  "architecture": {
    "network_class_name": "dynamic_network_architectures.architectures.unet.PlainConvUNet",
    "arch_kwargs": {
      "n_stages": 6,
      "features_per_stage": [32, 64, 128, 256, 320, 320],
      "kernel_sizes": [
        [1, 3, 3],
        [3, 3, 3]
      ],
      "strides": [
        [1, 1, 1],
        [1, 2, 2],
        [2, 2, 2]
      ]
    }
  }
}
```

The architecture dictionary is generated in:

`nnunetv2/experiment_planning/experiment_planners/default_experiment_planner.py:278`

3. How nnU-Net Selects the Network Shape

The experiment planner derives the network from:

- Voxel spacing
- Median image dimensions
- Dataset size

- Target GPU memory
- Whether the configuration is 2D or 3D

3.1 Patch Size

The initial patch aspect ratio is approximately inverse to voxel spacing:

```
tmp = 1 / np.array(spacing)
```

See `default_experiment_planner.py:249`.

Thick-slice data may therefore receive a patch such as `[64, 192, 192]`, rather than an isotropic patch. The planner repeatedly reduces the patch until its estimated feature-map memory fits the target VRAM budget. This search begins at `default_experiment_planner.py:300`.

3.2 Number of Stages

Stages are added while feature maps remain large enough to downsample. The topology algorithm is implemented in:

```
nnunetv2/experiment_planning/experiment_planners/network_topology.py:30
```

The first stage normally has stride 1. Later stages downsample eligible axes by a factor of 2.

3.3 Anisotropic Kernels and Strides

For anisotropic data, early stages may use:

```
kernel: (1, 3, 3)
stride: (1, 2, 2)
```

This avoids mixing information across thick slices and downsamples only the higher-resolution axes. When effective spacing becomes more isotropic, nnU-Net switches to:

```
kernel: (3, 3, 3)
stride: (2, 2, 2)
```

The kernel and stride selection logic is in `network_topology.py:52-108`.

4. Reading and Resolving Plans

The plans file is wrapped by two manager classes in:

```
nnunetv2/utilities/plans_handling/plans_handler.py
```

4.1 PlansManager

`PlansManager`, defined at `plans_handler.py:214`, handles:

- Loading the plans
- Listing available configurations
- Resolving configuration inheritance
- Selecting the image reader/writer
- Exposing axis transposition
- Constructing the label manager

A child configuration can reuse a parent configuration:

```
"3d_fullres_bs4": {
    "inherits_from": "3d_fullres",
    "batch_size": 4
}
```

Inheritance is resolved at `plans_handler.py:231`. The merge is shallow. If a child overrides `architecture`, it replaces the complete nested architecture dictionary rather than modifying only one nested key.

4.2 ConfigurationManager

ConfigurationManager, defined at `plans_handler.py:31`, provides direct access to the selected configuration:

```
configuration_manager.patch_size
configuration_manager.batch_size
configuration_manager.spacing
configuration_manager.network_arch_class_name
configuration_manager.network_arch_init_kwargs
```

This allows preprocessing and training code to consume a resolved configuration without repeatedly interpreting the raw JSON.

4.3 When Preprocessing Must Be Repeated

Preprocessing should generally be repeated when changing:

- Target spacing
- Normalization schemes
- Preprocessor class
- Resampling functions
- Any setting requiring a different `data_identifier`

Preprocessing usually does not need to be repeated for training-only changes such as batch size or architecture settings that reuse the same prepared data.

5. Network Construction

The trainer initializes the model in:

```
nnunetv2/training/nnUNetTrainer/nnUNetTrainer.py:209
```

The important call is conceptually:

```
self.network = self.build_network_architecture(
    plans_manager,
    configuration_manager,
    num_input_channels,
    num_output_channels,
    enable_deep_supervision,
)
```

The default implementation is at `nnUNetTrainer.py:349`. It delegates construction to:

```
nnunetv2/utilities/get_network_from_plans.py:9
```

`get_network_from_plans` performs the following steps:

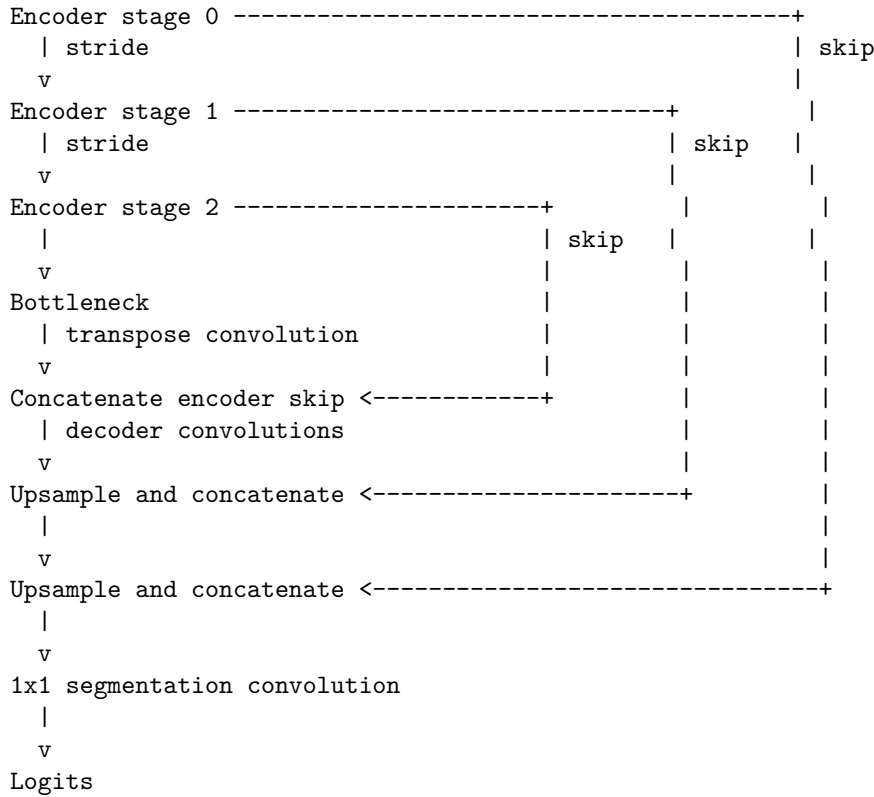
1. Copies the architecture keyword arguments from the plans.
2. Imports classes represented as strings, such as `Conv3d` and `InstanceNorm3d`.
3. Imports the requested architecture class.
4. Injects input channels, output channels, and the deep-supervision setting.
5. Instantiates the model.
6. Applies the architecture's weight initialization function.

The actual U-Net building blocks are supplied by the external `dynamic-network-architectures` package.

6. PlainConvUNet

The plain convolutional architecture follows the standard encoder-decoder pattern:

```
Input
|
v
```



6.1 Encoder Stage

For the default plain U-Net, each stage usually contains two convolution blocks:

```

Convolution
-> Instance normalization
-> LeakyReLU
-> Convolution
-> Instance normalization
-> LeakyReLU

```

Default layer settings are generated at `default_experiment_planner.py:278-298`:

- Conv2d or Conv3d
- Convolution bias enabled
- InstanceNorm2d or InstanceNorm3d
- LeakyReLU
- No dropout
- Usually two convolutions per encoder stage
- Usually two convolutions per decoder stage

Downsampling is normally performed by the first convolution of a stage using stride 2. A separate max-pooling layer is not required.

6.2 Feature Channels

The default feature count follows:

```
features = min(32 * 2**stage, maximum_features)
```

See `default_experiment_planner.py:234`.

A typical 3D network may therefore use:

```
32 -> 64 -> 128 -> 256 -> 320 -> 320
```

The default feature cap is commonly 320 for 3D and 512 for 2D.

6.3 Decoder Stage

Each decoder stage performs:

```
Transposed convolution
-> concatenate corresponding encoder skip
-> convolution blocks
-> segmentation head
```

The transposed convolution reverses the corresponding encoder stride. Each segmentation head is a 1x1 or 1x1x1 convolution that produces a tensor shaped approximately as:

```
[batch, segmentation_heads, spatial_dimensions...]
```

7. ResidualEncoderUNet

The residual encoder variant replaces plain encoder stages with residual blocks:

```

      +-----+ shortcut -----+
      |                                     |
Input -----+--> Conv -> Norm -> ReLU -> Conv -> Norm --+--> LeakyReLU

```

It uses:

- A separate stem convolution
- Residual encoder blocks
- The same general skip-connected U-Net decoder
- One decoder convolution per stage by default

Its planner is:

```
nnunetv2/experiment_planning/experiment_planners/residual_unets/residual_encoder_unet_planners.py
```

Default encoder block counts resemble:

```
[1, 3, 4, 6, 6, ...]
```

Residual presets M, L, and XL primarily vary network capacity, patch capacity, and target VRAM.

8. Input and Output Channels

8.1 Input Channels

For ordinary training:

```
input_channels = number of image modalities
```

Examples:

- CT: one input channel
- T1, T1ce, T2, and FLAIR: four input channels

For cascade training, one-hot channels from the previous-stage segmentation are appended to the image. Input-channel calculation is in:

```
nnunetv2/utilities/label_handling/label_handling.py:312
```

8.2 Output Channels

The label manager determines the number of segmentation heads:

- Class training: one head per class, including background
- Region training: one head per region
- Ignore labels do not create output heads

9. Deep Supervision

During training, the decoder can generate predictions at multiple resolutions:

```
[
    full-resolution logits,
    half-resolution logits,
    quarter-resolution logits,
    ...
]
```

The full-resolution output receives the largest loss weight. Coarser outputs receive exponentially smaller weights.

Deep-supervision scales are calculated from cumulative encoder strides in `nnUNetTrainer.py:383`. Loss weights are constructed at `nnUNetTrainer.py:451`.

During inference, deep supervision is disabled and the network returns only the full-resolution output.

10. Loss and Prediction

The network returns raw logits, not probabilities.

For mutually exclusive classes, the default objective is:

Dice loss + cross-entropy loss

For overlapping region training, the default objective is:

Dice loss + binary cross-entropy loss

Loss construction is in `nnUNetTrainer.py:436`.

Softmax or sigmoid is applied inside the loss or during final prediction conversion, not by the final network layer. Class predictions use argmax. Region predictions use sigmoid followed by thresholding.

11. Configuration-to-Layer Mapping

The most important architecture fields map to layers as follows:

Plan field	Effect on the network
<code>network_class_name</code>	Selects PlainConvUNet, ResidualEncoderUNet, or a custom class
<code>n_stages</code>	Number of encoder resolutions, including bottleneck
<code>features_per_stage</code>	Number of feature channels at each resolution
<code>conv_op</code>	Selects 2D or 3D convolution
<code>kernel_sizes</code>	Convolution kernel shape at each encoder stage
<code>strides</code>	Downsampling performed at each encoder stage
<code>n_conv_per_stage</code>	PlainConvUNet convolutions in each encoder stage
<code>n_blocks_per_stage</code>	Residual blocks in each residual encoder stage
<code>n_conv_per_stage_decoder</code>	Convolutions after skip concatenation in each decoder stage
<code>norm_op</code>	Normalization layer, normally instance normalization
<code>dropout_op</code>	Optional dropout layer; normally disabled
<code>nonlin</code>	Activation function, normally LeakyReLU
<code>deep_supervision</code>	Enables intermediate decoder segmentation outputs

12. Recommended Reading Order

1. `documentation/reference/plans-and-configuration.md`
2. `nnunetv2/experiment_planning/experiment_planners/default_experiment_planner.py:228`
3. `nnunetv2/experiment_planning/experiment_planners/network_topology.py:30`
4. `nnunetv2/utilities/plans_handling/plans_handler.py:31`
5. `nnunetv2/training/nnUNetTrainer/nnUNetTrainer.py:209`

6. `nnunetv2/utilities/get_network_from_plans.py:9`
7. The `PlainConvUNet`, `ResidualEncoderUNet`, and `UNetDecoder` implementations in `dynamic-network-architectures`

13. Compact Mental Model

Dataset characteristics

- > preprocessing choices
- > patch size and target spacing
- > stride and kernel layout
- > number of stages and feature channels
- > concrete PyTorch U-Net

nnU-Net's principal contribution is not a novel convolutional layer. Its strength is the systematic conversion of dataset properties and hardware constraints into a reliable segmentation pipeline and a dataset-adapted U-Net.